



UNIVERSIDADE FEDERAL DE GOIÁS
INSTITUTO DE INFORMÁTICA
BACHARELADO EM ENGENHARIA DE SOFTWARE
PHABLO TAVARES PAIXÃO

Evolução e Refatoração Full-Stack do Sistema SIPROS

**Segurança, Qualidade e Melhorias de Interface em uma
Fábrica de Software**

Goiânia
2026

PHABLO TAVARES PAIXÃO

Evolução e Refatoração Full-Stack do Sistema SIPROS

Segurança, Qualidade e Melhorias de Interface em uma Fábrica de Software

Trabalho de Conclusão apresentado à Coordenação do Curso de Engenharia de Software do Instituto de Informática da Universidade Federal de Goiás, como requisito parcial para obtenção do título de Bacharel em Engenharia de Software.

Orientadora: Profa. Sofia Larissa da Costa Paiva

Co-Orientador: Prof. Juliano Lopes de Oliveira

Goiânia
2026

Todos os direitos reservados. É proibida a reprodução total ou parcial do trabalho sem autorização da universidade, do autor e do orientador(a).

Phablo Tavares Paixão

Graduando em Engenharia de Software pela Universidade Federal de Goiás (UFG). Durante sua trajetória acadêmica, atuou como desenvolvedor back-end no Centro de Excelência em Empreendedorismo Tecnológico (CETT/UFG) e foi bolsista de pesquisa no Centro de Excelência em Inteligência Artificial (CEIA/UFG). Possui experiência em engenharia de software e desenvolvimento mobile (Flutter) pela Personal Fit. Atualmente, atua como Analista de Inteligência Artificial na Rennova, desenvolvendo soluções de automação de processos e integração de inteligência artificial generativa (LLMs).

Agradecimentos

Agradeço à coordenação da Fábrica de Software pelo apoio e trabalho em conjunto, nos guiando ao longo do semestre e nos proporcionando a inestimável oportunidade de atuar em um projeto real desse calibre.

Agradeço à professora Sofia Larissa da Costa Paiva e ao professor Juliano Lopes de Oliveira pelo apoio imenso e orientação fundamentais nesta jornada.

Agradeço aos meus colegas da Equipe Dourada pela dedicação e pelo grande aprendizado que construímos em conjunto durante o desenvolvimento do SIPROS.

Resumo

Paixão, Phablo Tavares. **Evolução e Refatoração Full-Stack do Sistema SI-PROS**. Goiânia, 2026. 39p. Relatório de Graduação. Bacharelado em Engenharia de Software, Instituto de Informática, Universidade Federal de Goiás.

Contexto: A formação acadêmica em Engenharia de Software requer vivência prática para a consolidação do aprendizado teórico. Atuar em ambientes que simulam a indústria, como uma Fábrica de Software, proporciona uma transição realista, exigindo a adaptação a arquiteturas existentes, a garantia contínua da qualidade do produto e a adoção de protocolos de segurança modernos.

Objetivo: Este trabalho tem como objetivo descrever e analisar a experiência de evolução e refatoração full-stack do Sistema Integrado de Processos Seletivos (SI-PROS) da UFG, destacando a implementação de mecanismos robustos de segurança, melhorias na interface de usuário e a adoção de práticas de garantia da qualidade.

Método: O trabalho foi desenvolvido sob a metodologia de pesquisa-ação no contexto da disciplina de Fábrica de Software no semestre 2026/1. Utilizaram-se métodos ágeis para o planejamento e desenvolvimento iterativo. A execução envolveu processos rigorosos de Verificação e Validação (V&V) entre Casos de Uso e protótipos, além da aplicação sistemática de revisões por pares (*Code Review*) e testes automatizados.

Resultados: A intervenção resultou na integração bem-sucedida da autenticação do Google via Keycloak, adotando a extensão PKCE (*Proof Key for Code Exchange*) para mitigar vulnerabilidades estruturais no *front-end* em Angular. Simultaneamente, as práticas de V&V permitiram a correção de dezenas de inconsistências de arquitetura e usabilidade, enquanto a estruturação formal da suíte de testes de unidade garantiu a integridade do *back-end* contra regressões.

Conclusões: Os resultados evidenciam que a implementação de protocolos de segurança em Aplicações de Página Única (SPAs) exige um rigor arquitetural que afeta todo o fluxo do sistema. A experiência atestou que o emprego constante de práticas de qualidade (testes e V&V) é indispensável em projetos colaborativos. Por fim, a atuação na Fábrica de Software proporcionou um amadurecimento técnico e comportamental inestimável, aproximando genuinamente a vivência acadêmica da realidade do mercado de tecnologia.

Palavras-chave

Engenharia de Software; Fábrica de Software; Autenticação e Segurança; Qualidade de Software; Desenvolvimento Full-Stack.

Abstract

Paixão, Phablo Tavares. **Full-Stack Evolution and Refactoring of the SIPROS System: Security, Quality, and Interface Improvements in a Software Factory**. Goiânia, 2026. 39p. Relatório de Graduação. Bacharelado em Engenharia de Software, Instituto de Informática, Universidade Federal de Goiás.

Context: Academic training in Software Engineering requires practical experience to consolidate theoretical learning. Working in environments that simulate the industry, such as an academic Software Factory, provides a realistic transition, demanding adaptation to existing architectures, continuous quality assurance, and the adoption of modern security protocols.

Objective: This work aims to describe and analyze the practical experience of the full-stack evolution and refactoring of the Integrated Selection Processes System (SIPROS) at UFG, highlighting the implementation of robust security mechanisms, user interface improvements, and the adoption of quality assurance practices.

Method: The work was developed using action research methodology within the context of the Software Factory course during the 2026/1 semester. Agile methods were utilized for planning and iterative development. The execution involved rigorous Verification and Validation (V&V) processes between Use Cases and prototypes, as well as the systematic application of peer reviews (Code Review) and automated testing.

Results: The intervention resulted in the successful integration of Google authentication via Keycloak, adopting the PKCE (Proof Key for Code Exchange) extension to mitigate structural vulnerabilities in the Angular front-end. Simultaneously, the V&V practices allowed the correction of dozens of architectural and usability inconsistencies, while the formal structuring of the unit test suite ensured the back-end's integrity against regressions.

Conclusions: The results show that implementing security protocols in Single Page Applications (SPAs) requires architectural rigor that affects the entire system flow. The experience confirmed that the constant employment of quality practices (testing and V&V) is indispensable in collaborative projects. Finally, working in the Software Factory provided invaluable technical and behavioral maturation, genuinely bringing the academic experience closer to the reality of the technology market.

Keywords

Software Engineering; Software Factory; Authentication and Security; Software Quality; Full-Stack Development.

Sumário

Lista de Figuras	10
Lista de Tabelas	11
Lista de Algoritmos	12
Lista de Códigos de Programas	13
1 Introdução	14
1.1 Motivação e Justificativa	14
1.2 Contexto da Experiência	14
1.3 Objetivo do Trabalho	15
1.3.1 Objetivo Geral	15
1.3.2 Objetivos Específicos	15
1.4 Metodologia do Trabalho	16
1.5 Organização do Trabalho	16
2 Bases Teóricas e Tecnológicas	18
2.1 Tópico Específico trabalhado no TCC	18
2.2 Processos de Ciclo de Vida de Software	19
2.3 Organização do Trabalho Colaborativo em Software	19
2.4 Ferramentas e Tecnologias Adotadas	19
3 Relato de Experiência	21
3.1 Contextualização do Projeto	21
3.1.1 Ambiente Organizacional	21
3.1.2 Forma de Participação no Projeto	22
3.2 Atividades Realizadas e Resultados Gerados	22
3.3 Integração com a Equipe e Processo de Trabalho	22
3.4 Desafios Enfrentados	22
3.5 Soluções Adotadas e Resultados Obtidos	23
3.6 Avaliação de Qualidade de Software	23
3.7 Aprendizados e Desenvolvimento Profissional	23
4 Conclusões	24
4.1 Considerações Finais	24
4.2 Síntese da Experiência na Fábrica de Software	24
4.3 Trabalhos Relacionados	24
4.4 Trabalhos Futuros	25

Referências	26
A Como usar este modelo para elaborar seu TCC	27
A.1 Introdução	27
A.2 Descrição da classe inf-ufg	28
A.2.1 Opções da classe	28
A.2.2 Parâmetros da classe	28
A.2.3 Elementos Pré-Textuais	30
A.3 Elementos do texto	31
A.3.1 Figuras	31
A.3.2 Subfiguras	34
A.3.3 Tabelas	35
A.3.4 Algoritmos	35
A.3.5 Códigos de Programa	36
A.3.6 Teoremas, Corolários e Demonstrações	37
A.3.7 Citações Longas	38
A.3.8 Referências Bibliográficas	38

Lista de Figuras

A.1	Uma figura típica.	33
A.2	Esta figura é um exemplo de um rótulo de figura que ocupa mais de uma linha, devendo ser indentado e justificado.	33
A.3	Figura incluída no texto com a classe <code>graphicx</code> .	34
A.4	(a) e (b) representam dois exemplos do uso de subfiguras dentro de uma única figura.	34
	(b) Segunda subfigura (um pedaço).	34

Lista de Tabelas

A.1 Conteúdo do diretório

36

Lista de Algoritmos

A.1 $MSR(A, i, j)$

36

Lista de Códigos de Programas

A.1 `insertionsort()`

37

Introdução

1.1 Motivação e Justificativa

A sociedade contemporânea apresenta uma dependência crescente em relação a sistemas de software, exigindo que aplicações, especialmente em contextos institucionais e governamentais, sejam desenvolvidas com alto rigor arquitetural e foco em segurança. Plataformas acadêmicas lidam diariamente com dados sensíveis e necessitam de processos eficientes de desenvolvimento e manutenção contínua.

O Sistema Integrado de Processos Seletivos (SIPROS) da Universidade Federal de Goiás (UFG) foi concebido para centralizar e padronizar editais de seleção. Contudo, em seu estado prévio ao semestre letivo de 2026/1, o sistema apresentava dívidas técnicas significativas: fluxos simulados (mockados), rotas desconexas, credenciais embutidas diretamente no código-fonte e ausência de uma camada de autenticação robusta.

Atuar na resolução desses problemas em um ambiente colaborativo é fundamental. O trabalho em equipe em projetos de software reais traz desafios organizacionais e técnicos que não podem ser simulados em projetos acadêmicos teóricos isolados. A vivência e a aplicação de boas práticas de Engenharia de Software, em um cenário legado e em constante evolução, proporcionam um ganho prático substancial. O relato desta experiência justifica-se por sua valiosa contribuição acadêmica e prática, detalhando como problemas críticos de segurança e consistência de interface podem ser resolvidos. Para estudantes e futuras turmas, este documento serve como um guia de lições aprendidas; para a coordenação da Fábrica de Software, documenta a evolução arquitetural do próprio SIPROS.

1.2 Contexto da Experiência

O presente relato se insere no contexto da disciplina de Prática em Engenharia de Software, ofertada pelo Instituto de Informática da UFG durante o semestre letivo de

2026/1. A Fábrica de Software acadêmica tem como propósito simular o ambiente de mercado e propõe aos discentes a manutenção evolutiva e refatoração de sistemas reais.

O autor do trabalho integrou a "Equipe Dourada", composta por um total de sete discentes, responsável por atuar no desenvolvimento *full-stack* do sistema SIPROS. No que tange à organização do processo de desenvolvimento, a equipe não esteve obrigada a adotar de forma estrita nenhum *framework* ágil específico. Em vez disso, adotou-se uma metodologia de desenvolvimento **inspirada** nas metodologias ágeis e no *Scrum*, incorporando suas principais características (como as *Sprints*), mas sendo adaptada continuamente para o contexto e necessidades da equipe ao longo do semestre. A coordenação da Fábrica atuou em um papel semelhante ao de um *Product Owner*, definindo as demandas e prioridades, enquanto os discentes possuíam autonomia para o autogerenciamento, divisão de tarefas e aplicação de ferramentas visuais, como quadros *Kanban*.

1.3 Objetivo do Trabalho

Este Trabalho de Conclusão de Curso possui um objetivo principal que delinea a pesquisa, sendo o mesmo desdobrado em metas específicas que orientam a descrição dos resultados obtidos.

1.3.1 Objetivo Geral

O objetivo principal deste trabalho é descrever e analisar a experiência prática de evolução e refatoração *full-stack* do sistema SIPROS no contexto da Fábrica de Software acadêmica da UFG, com ênfase na implementação de mecanismos robustos de segurança, autenticação e na adoção de práticas visando a elevação da qualidade do software.

1.3.2 Objetivos Específicos

Para alcançar o objetivo geral traçado, foram definidos os seguintes objetivos específicos:

1. Descrever o ambiente organizacional, a metodologia de trabalho inspirada em métodos ágeis e a dinâmica de colaboração adotada no projeto SIPROS.
2. Relatar as implementações técnicas voltadas à segurança da informação, destacando a integração com o provedor de identidade do Google via Keycloak, utilizando a extensão PKCE.

3. Descrever a execução dos processos de Verificação e Validação (V&V) aplicados para garantir a conformidade arquitetural da interface (*front-end*) com os Casos de Uso estabelecidos.
4. Apresentar as práticas de Garantia de Qualidade (QA) adotadas pela equipe, evidenciando a estruturação da suíte de testes automatizados e a adoção obrigatória das revisões por pares (*Code Review*).
5. Refletir criticamente sobre os desafios e gargalos vivenciados, analisando o amadurecimento e a evolução nas *hard skills* e *soft skills* proporcionados por essa vivência de caráter profissional.

1.4 Metodologia do Trabalho

No âmbito metodológico acadêmico, a elaboração do presente trabalho baseia-se na metodologia de **pesquisa-ação**. Diferentemente de um estudo de caso puramente observacional, a pesquisa-ação caracteriza-se pela intervenção ativa do pesquisador no próprio ambiente para a resolução de um problema real de Engenharia de Software. Neste cenário, o autor atuou ativamente como membro da equipe de desenvolvimento, mapeando inconsistências, projetando soluções de *design* arquitetural e efetuando modificações no código-fonte, participando diretamente da resolução do problema do sistema legado e na consequente implantação de um padrão de segurança seguro e atual.

Para fundamentar as atividades realizadas, a pesquisa se apoiou em literatura técnico-científica reconhecida na área, como o corpo de conhecimento da Engenharia de Software (SWEBOK) para os conceitos de Verificação e Validação, e normas internacionais como a ISO/IEC 25010 para modelagem de qualidade de produto. Complementarmente, consultou-se a documentação técnica pertinente (RFCs do protocolo OAuth 2.0 e manuais oficiais do *Keycloak* e do ecossistema *Angular*) e utilizou-se do acervo de artefatos produzidos internamente pelo projeto (*Issues*, *Pull Requests* e documentação de Casos de Uso).

Ressalta-se que a metodologia de desenvolvimento de software utilizada no dia a dia da equipe — inspirada no paradigma ágil — distingue-se desta metodologia acadêmica de pesquisa-ação e será tratada em maiores detalhes no Capítulo 3.

1.5 Organização do Trabalho

Para organizar o fluxo lógico do documento e guiar a leitura do relato, este trabalho encontra-se estruturado da seguinte forma:

- Capítulo 2: Apresenta as bases teóricas, conceituais e tecnológicas requeridas na execução das tarefas, cobrindo tópicos de Qualidade de Software, Verificação e Validação, além das premissas de Segurança em Aplicações Web baseadas em tokens (OAuth 2.0 e PKCE).
- Capítulo 3: Concentra a descrição detalhada do relato de experiência do discente. Expõe os desafios enfrentados no *onboarding*, as tomadas de decisões arquiteturais na integração com o GCP (*Google Cloud Platform*), o rastreamento e mitigação de defeitos e os resultados consolidados na melhoria do SIPROS.
- Capítulo 4: Corresponde às considerações finais, contendo a reflexão crítica do autor sobre a evolução das competências necessárias ao Engenheiro de Software, um panorama dos Trabalhos Futuros para o projeto e as percepções consolidadas a partir do semestre na Fábrica de Software.

Bases Teóricas e Tecnológicas

Este capítulo deve descrever a base conceitual do TCC. Explicar os pilares teóricos específicos que sustentam o projeto, como por exemplo: arquitetura de software, requisitos, ciclo de vida, etc.

Apresente uma visão geral do tópico trabalhado no TCC, sem tentar falar de Engenharia de Software como um todo. Pode falar do ciclo de vida de processos de software, mas o foco deve ser somente no que foi **efetivamente trabalhado e realizado no seu próprio TCC**.

O capítulo também deve fazer uma síntese dos principais conceitos instrumentais (ferramentas e tecnologias) aplicados ao longo do TCC, destacando como eles ajudam a compreensão da experiência relatada neste documento.

Este capítulo deve estabelecer uma conexão clara entre a teoria utilizada, as ferramentas aplicadas e as atividades práticas desenvolvidas, preparando o leitor para o capítulo seguinte, que é o próprio relato da experiência.

2.1 Tópico Específico trabalhado no TCC

Apresente os conceitos fundamentais do tópico específico da Engenharia de Software que foi trabalhado no TCC, destacando sua importância para projetos de Engenharia de Software que precisam utilizar métodos, práticas e técnicas para o desenvolvimento e a evolução de sistemas computacionais.

Coloque no título desta seção o tópico específico da Engenharia de Software que foi o assunto principal trabalhado no TCC. Por exemplo: Teste de Software; Requisitos de Software; Integração de Software; Técnicas de Validação de Software; Implementação de Interfaces Humano-Computador; Implementação de Funções de *Backend* de Software. Se trabalhou em mais de um tópico, crie uma subseção para cada tópico, e não misture dois ou mais assuntos diferentes na mesma seção.

Discuta os objetivos do tópico trabalhado, seus princípios e os desafios inerentes à criação e manutenção de software de qualidade. A seção deve estabelecer a base

conceitual necessária para compreender qualquer atividade realizada no contexto do projeto descrito no trabalho.

2.2 Processos de Ciclo de Vida de Software

Aqui são apresentados os principais processos de Engenharia de Software relacionados ao tópico trabalhado. Por exemplo, processos de Verificação e Validação, se o tópico é Teste de Software; ou processos de Planejamento e Monitoramento de projeto, se o tópico é Gerência de Projeto de Software.

São discutidas as características de cada processo, suas vantagens e limitações, bem como sua aplicabilidade em diferentes contextos. Esta fundamentação permite situar a atividade realizada dentro de um processo mais amplo de ciclo de vida de software, que contempla o desenvolvimento e a manutenção de software.

Cada processo deve ser apresentado de forma conceitual, indicando suas atividades genéricas e permitindo relacionar a teoria de Engenharia de Software com as tarefas específicas desempenhadas durante a experiência prática, independentemente da natureza dessas tarefas.

2.3 Organização do Trabalho Colaborativo em Software

Discuta os aspectos relacionados ao trabalho em equipe no contexto de projetos de software, considerando especificamente o papel ou função desempenhado durante o trabalho de TCC.

Aborde temas como comunicação, coordenação, divisão de responsabilidades, interfaces com outros papéis típicos em equipes técnicas e desafios comuns enfrentados por equipes de desenvolvimento relacionadas ao papel desempenhado.

Esta seção deve fundamentar a sua análise da experiência ao participar de uma equipe real, destacando fatores humanos e organizacionais que influenciaram sua atuação no projeto.

2.4 Ferramentas e Tecnologias Adotadas

Apresente e analise as ferramentas utilizadas para apoiar os processos e atividades discutidas anteriormente.

As ferramentas devem ser comparadas a outras amplamente utilizadas na indústria de software, justificando sua escolha para o trabalho realizado.

Evite fazer apenas uma propaganda das ferramentas. Discuta os pontos fortes, mas analise também as limitações de cada ferramenta. Devem ser discutidos artefatos

gerados nas atividades dos processos e a forma como as ferramentas apoiam a criação e evolução destes artefatos.

A discussão de ferramentas permite contextualizar a forma como o trabalho da equipe e do autor do TCC foi organizado e como as atividades do autor se inseriram nos processos colaborativos da Engenharia de Software, conforme discutido na seção anterior.

Relato de Experiência

3.1 Contextualização do Projeto

Explique inicialmente por que o software envolvido no projeto é relevante: que problema ele resolve, para que público-alvo e com que nível de criticidade para as partes interessadas no software.

Explique, em seguida, o cenário de cada projeto em que atuou: qual problema existia, por que o software foi criado ou modificado no projeto, ou seja, qual a finalidade do projeto, e quem são as partes interessadas no projeto.

Sintetize, ainda, a **metodologia de desenvolvimento ou manutenção de software** usada pela equipe de Engenharia de Software do projeto (Scrum, Kanban etc.).

3.1.1 Ambiente Organizacional

Descreva o ambiente organizacional em que o projeto foi realizado, em termos de cultura organizacional, abordagem de processos (definidos ou informais, rigorosos ou ágeis), local e infraestrutura de trabalho, pessoas envolvidas e seus respectivos papéis, forma de trabalho (presencial, remoto ou híbrido), dimensão da equipe, entre outras características relevantes para compreender o ambiente em que o projeto se desenvolveu.

Explique sucintamente como o projeto foi organizado em termos de **metodologia de desenvolvimento ou manutenção de software** usada pela equipe de Engenharia de Software do projeto (Scrum, Kanban, XP, RUP, etc.), mostrando os macro-processos.

Discuta sucintamente as principais ferramentas, tecnologias e *frameworks* (Git, Figma, Docker, etc.) aplicados no projeto.

Descreva, ainda, os papéis e responsabilidades da equipe, principalmente aqueles que tinham interface ou interação com o papel que você exerceu.

3.1.2 Forma de Participação no Projeto

Apresente o contexto geral da sua participação no(s) projeto(s) de software. Retome o tipo de projeto (desenvolvimento, evolução, adaptação, correção, migração, etc.) e o seu(s) papel(is) na equipe envolvida.

Explique como você passou a integrar o(s) projeto(s) e quais eram as expectativas iniciais em relação às atividades a serem desempenhadas.

Explique, ainda, como você se familiarizou com o projeto antes de começar a contribuir efetivamente de acordo com o seu papel no projeto.

3.2 Atividades Realizadas e Resultados Gerados

Detalhe as atividades específicas que você desempenhou ao longo do projeto, tais como engenharia de requisitos, modelagem, projeto de software, implementação, testes, documentação de processos, manutenção corretiva ou evolutiva, entre outras.

A descrição deve evidenciar como cada atividade foi atribuída a você e como foi conduzida por você: quais ferramentas ou técnicas foram utilizadas, resultados gerados e como estes contribuíram para o avanço do projeto.

Descreva o que foi feito, como foi feito e por quê.

3.3 Integração com a Equipe e Processo de Trabalho

Apresente a forma como você interagiu com os demais membros da equipe. Discuta aspectos como comunicação, colaboração, reuniões, uso de ferramentas de gestão e práticas adotadas pelo grupo.

Descreva também como o processo de trabalho da equipe influenciou as suas atividades, considerando metodologias, rotinas e dinâmicas internas.

3.4 Desafios Enfrentados

Discuta os principais desafios encontrados durante a sua experiência na Fábrica de Software. Os desafios podem ser das mais diversas naturezas, tais como técnicos, organizacionais, comunicacionais, relacionados ao aprendizado de novas ferramentas e práticas ou ao desenvolvimento de habilidades pessoais e comportamentais.

A análise deve destacar como esses desafios impactaram o andamento das suas atividades e quais estratégias foram utilizadas para superá-los.

3.5 Soluções Adotadas e Resultados Obtidos

Discuta as soluções implementadas para lidar com os desafios identificados e os resultados alcançados com as atividades realizadas. Os resultados podem incluir entregas concluídas, melhorias no processo, correções implementadas, artefatos produzidos, habilidades adquiridas ou aprimoradas, ou qualquer outro impacto positivo decorrente da sua participação no(s) projeto(s).

Analise o impacto do trabalho para as partes interessadas: o que melhorou, o que funcionou, o que não funcionou. Analise criticamente a evolução do produto e a avaliação das partes interessadas, tanto usuários quanto outros *stakeholders*, em comparação com os objetivos iniciais do projeto.

3.6 Avaliação de Qualidade de Software

Apresente os conceitos relacionados à qualidade de software das atividades que você realizou, incluindo critérios de qualidade, indicadores e métricas usadas na avaliação, estratégias de verificação e validação e práticas de garantia de qualidade aplicadas ao trabalho realizado no TCC.

Fundamente a qualidade dos artefatos que você gerou durante o TCC com base em evidências produzidas por atividades como testes, revisão de código, *walkthrough* de documentação ou qualquer outra ação voltada à avaliação da qualidade do produto ou do processo de software.

3.7 Aprendizados e Desenvolvimento Profissional

Discuta os aprendizados adquiridos ao longo da experiência, dos pontos de vista técnico, profissional e pessoal.

Devem ser abordados conhecimentos consolidados, habilidades (*hard* e *soft*) desenvolvidas, boas práticas assimiladas e reflexões sobre o trabalho em equipe e sobre o processo de engenharia de software.

Analise, com destaque, se, e como, a experiência realizada na Fábrica de Software contribuiu para a sua formação integral como profissional e como cidadão.

Conclusões

4.1 Considerações Finais

Retomar e resumir, sucintamente, a motivação do trabalho, bem como a metodologia, os resultados obtidos e as contribuições alcançadas por este trabalho de TCC.

Discuta especificamente o **alcance dos objetivos** geral e específicos.

4.2 Síntese da Experiência na Fábrica de Software

Apresente uma síntese da experiência vivenciada, destacando os aspectos mais relevantes do processo, os principais resultados e a importância da participação no projeto para o desenvolvimento acadêmico e profissional do aluno.

Discutir as principais lições aprendidas, sintetizando respostas para as seguintes perguntas:

1. Quais são as suas impressões sobre a experiência realizada de prática de Engenharia de Software em um projeto real da Fábrica de Software?
2. Quais os principais aprendizados, conquistas e evoluções em competências pessoais/comportamentais (*soft skills*) e técnicas (*hard skills*) obtidos da experiência realizada?
3. Há outros benefícios associados à esta experiência, além de melhoria de competências *soft* e *hard*?
4. Quais as principais dificuldades enfrentadas na experiência prática vivenciada?
5. Quais sugestões você daria para estudantes que irão vivenciar esta experiência nos próximos semestres?

4.3 Trabalhos Relacionados

Esta seção deve referenciar estudos, artigos, relatos de experiência e outros trabalhos acadêmicos que abordam temas semelhantes ao deste TCC.

Deve ser feita uma análise comparativa entre os trabalhos referenciados e o presente documento de TCC. A comparação com trabalhos existentes permite situar o presente relato no contexto da literatura, identificar contribuições relevantes e confirmar ou refutar resultados e hipóteses apresentadas em outros trabalhos.

4.4 Trabalhos Futuros

Discuta as limitações do presente trabalho, indicando como futuros trabalhos podem reduzir essas limitações. Também aponte possíveis melhorias ou perspectivas para trabalhos futuros que complementem os resultados gerados neste TCC. Por exemplo, indique potenciais melhorias no produto de software que foi alvo dos processos de desenvolvimento ou manutenção, possíveis melhorias nesses processos, ou ainda possíveis aplicações de ferramentas ou tecnologias que poderiam melhorar o produto ou o processo de Engenharia de Software.

Referências

- [ABNT, 2001a] ABNT (2001a). *NBR 10520*. Associação Brasileira de Normas Técnicas, Rio de Janeiro.
- [ABNT, 2001b] ABNT (2001b). *NBR 14724*. Associação Brasileira de Normas Técnicas, Rio de Janeiro.
- [Berge, 1970] Berge, C. (1970). *Graphes et hypergraphes*. Paris: Dunod.
- [Berners-Lee & Swick, 2002] Berners-Lee, R. & Swick, T. (2002). Semantic Web Development.
- [Drakos, 1994] Drakos, N. (1994). *The L^AT_EX to HTML translator*. Internal report, Computer Based Learning Unit, University of Leeds.
- [Goossens et al., 1994] Goossens, M., Mittelbach, F., & Samarin, A. (1994). *The L^AT_EX Companion*. Reading, MA, USA: Addison-Wesley, second edition.
- [Hahn, 1991] Hahn, J. (1991). *L^AT_EX for Everyone*. 12 Madrona Street, Mill Valley, CA 94941, USA: Personal T_EX Inc.
- [Knuth, 1989] Knuth, D. E. (1989). *The TeX Book*. Addison-Wesley, 15th edition.
- [Kopka & Daly, 1995] Kopka, H. & Daly, P. W. (1995). *A Guide to L^AT_EX2_ε: Document Preparation for Beginners and Advanced Users*. Reading, MA, USA: Addison-Wesley, second edition.
- [Lamport, 1996] Lamport, L. (1996). *L^AT_EX: A Document Preparation System*. Reading, MA, USA: Addison-Wesley, second edition.
- [Lewiner, 2002] Lewiner, T. (2002). *Normas para apresentação de teses e dissertações*. Technical report, Departamento de Matemática - PUC-Rio.
- [Santini Frasson, 2002] Santini Frasson, M. V. (2002). *Classe ABNT*. Grupo abnTeX.
- [Smith & Jones, 1999] Smith, A. & Jones, B. (1999). On the Complexity of Computing. In A. B. Smith-Jones (Ed.), *Advances in Computer Science* (pp. 555–566). Publishing Press.

Como usar este modelo para elaborar seu TCC

A.1 Introdução

$\text{\LaTeX}$ é um sistema de editoração eletrônica muito usado para produzir documentos científicos de alta qualidade tipográfica. Este apêndice mostra como usar o $\text{\LaTeX}$ com a classe `inf-ufg` para formatar teses, dissertações, monografias e relatórios de conclusão de curso, segundo o padrão adotado pelo Instituto de Informática da UFG. Este documento e a classe `inf-ufg` foram, em grande parte, copiados e adaptados da classe `thesisPUC` e do texto de Thomas Lewiner [Lewiner, 2002] que descreve a sua utilização.

Se você precisar de material de apoio referente ao $\text{\LaTeX}$, veja um dos sites do Comprehensive TEX Archive Network (CTAN) em www.ctan.org. Todos os pacotes podem ser obtidos via FTP <ftp://www.ctan.org> e existem vários servidores em todo o mundo. Eles podem ser encontrados, por exemplo, em <ftp://ctan.tug.org> (EUA), <ftp://ftp.dante.de> (Alemanha), <ftp://ftp.tex.ac.uk> (Reino Unido).

Você pode encontrar muitas informações e dicas na página dos usuários brasileiros de $\text{\LaTeX}$ ($\text{\TeX}$ -BR). Em particular no CTAN, está disponível uma introdução bastante completa em português: CTAN:/tex-archive/info/lshort/portuguese-BR/.

O ambiente Overleaf (<https://pt.overleaf.com>) fornece uma plataforma web completa para processar textos em $\text{\LaTeX}$ sem necessidade de instalar qualquer tipo de software em seu computador. Muitas dicas interessantes sobre $\text{\LaTeX}$ podem ser obtidas em: <https://pt.overleaf.com/learn>.

Todavia, se você quer usar o $\text{\LaTeX}$ em seu computador, verifique em quais sistemas ele está disponível em CTAN:/tex-archive/systems. Em particular para MS Windows, o sistema gratuito MikTeX, disponível no CTAN e no site www.miktex.org tem muitas opções que você poderia precisar para editar o seu texto.

A.2 Descrição da classe inf-ufg

O estilo inf-ufg se integra completamente ao $\text{\LaTeX 2}_{\epsilon}$. O estilo inf-ufg foi desenhado para minimizar a quantidade de texto e de comandos necessários para escrever a sua dissertação. Só é preciso inserir algumas macros no início do seu arquivo $\text{\LaTeX}$, precisando os dados bibliográficos da sua dissertação (por exemplo o seu nome, o título da dissertação...). Em seguida, cada página dos elementos pré-textuais será formatada usando macros ou ambientes específicos. O corpo do texto é editado normalmente. Finalmente, as referências bibliográficas podem ser entradas manualmente (via o comando `\bibitem` do $\text{\LaTeX}$ padrão) ou usando o sistema BiBTeX (muito mais recomendável). Neste caso, os arquivos `inf-ufg.bst` e `abnt-alf.bst` permitem a formatação das referências bibliográficas segundo as normas da UFG.

A.2.1 Opções da classe

Para usar esta classe num documento $\text{\LaTeX 2}_{\epsilon}$, coloque os arquivos `inf-ufg.cls`, `inf-ufg.bst`, `abnt-num.bst`, `atbeginend.sty` e `tocloft.sty` numa pasta onde o compilador $\text{\LaTeX}$ pode achá-lo (normalmente na mesma pasta que seu arquivo `.tex`), e defina-o como o estilo do seu documento. Por exemplo, uma dissertação de mestrado que usa o modelo abnt de citações bibliográficas:

```
\documentclass[dissertacao,abnt]{inf-ufg}
...
\begin{document}
```

As opções da classe são `[tese]` (para tese de doutorado), `[dissertacao]` (para dissertação de mestrado), `[monografia]` (para monografia de curso de especialização) e `[relatorio]` (para relatório final de curso de graduação). Se nenhuma opção for declarada, o documento é considerado como uma dissertação de mestrado. Se a opção `[abnt]` for utilizada, as citações bibliográficas serão geradas conforme definido pelo grupo de trabalho `abnt-tex`. Contudo, o mais recomendável é não utilizar essa opção. Com a opção `[nocolorlinks]` todos os *links* de navegação no texto ficam na cor preta. O ideal é usar esta opção para gerar o arquivo para impressão, pois a qualidade da impressão dos *links* fica superior.

A.2.2 Parâmetros da classe

Os elementos pré-textuais são definidos página por página e dependem da correta definição dos parâmetros listados a seguir (aqueles que contêm um texto/valor padrão não precisam ser definidos, caso atenda a situação do autor do texto que está usando a classe `inf-ufg.cls`):

- `\autor` : Nome completo do autor da tese, começando pelo apelido (ex.: José da Silva);
- `\autorR` : Nome completo do autor da tese, começando pelo nome (ex.: da Silva, José);
- `\titulo` : Título da tese, dissertação, monografia ou relatório de conclusão de curso;
- `\subtitulo` : Se tiver um subtítulo, use este macro para defini-lo;
- `\cidade` : A cidade de edição. A cidade padrão é Goiânia.
- `\dia` : Dia do mês da data de defesa (1–31);
- `\mes` : Mês da data de defesa (1–12);
- `\ano` : Ano da data de defesa;
- `\universidade` : Nome completo da universidade. O nome padrão é Universidade Federal de Goiás;
- `\uni` : Sigla da universidade. A sigla padrão é UFG;
- `\unidade` : Nome da unidade acadêmica. O padrão é Instituto de Informática;
- `\departamento` : Nome do departamento, com maiúscula na primeira letra (para o caso de unidades com mais de um departamento);
- `\programa` : Nome do programa de pós-graduação, com maiúscula na primeira letra. O padrão é Computação;
- `\concentracao` : Nome da área de concentração;
- `\orientador` : Nome completo do orientador, começando pelo apelido;
- `\orientadorR` : Nome completo do orientador, começando pelo nome;
- `\orientadora` : Nome completo da orientadora, começando pelo apelido; use este comando e o próximo se for orientadora e não orientador.
- `\orientadoraR` : Nome completo da orientadora, começando pelo nome;
- `\coorientador` : Nome completo do co-orientador, começando pelo apelido;
- `\coorientadorR` : Nome completo do co-orientador, começando pelo nome;
- `\coorientadora` : Nome completo da coorientadora, começando pelo apelido; use este comando e o próximo se for coorientadora e não coorientador.
- `\coorientadoraR` : Nome completo da coorientadora, começando pelo nome;
- `\universidadeco` : Nome da universidade do coorientador;
- `\unico` : Sigla da universidade do coorientador;
- `\unidadeco` : Nome da unidade acadêmica do coorientador.¹

¹Se não tiver um co-orientador, não defina esses últimos sete parâmetros.

A.2.3 Elementos Pré-Textuais

Os elementos pré-textuais são definidos página por página, conforme descritos a seguir:

capa

`\capa` : Gera o modelo da capa externa do trabalho. Esta página servirá apenas como modelo para a encadernação da versão final do texto. Nenhum dado é necessário.

publicação

`\publica` : Gera a autorização para publicação do trabalho em formato eletrônico e disponibilização do mesmo na biblioteca virtual da UFG.

rosto

`\rosto` : Gera a folha de rosto, a qual é a primeira folha interna do trabalho. Nenhum dado é necessário.

aprovação

`\aprovacao` : ambiente para a reprodução do termo de aprovação da Banca Examinadora da tese ou dissertação.

banca

`\banca` : Entrada para o nome dos examinadores, exceto o(s) orientador(es).
`\profa` : Entrada para o nome das examinadoras, exceto o(s) orientador(es).

direitos

`\direitos` : Macro com 2 argumentos para gerar os direitos autorais, o perfil do aluno e a ficha catalográfica da Biblioteca Central da UFG.

- O primeiro argumento é o Perfil do aluno; e
- O segundo argumento é a lista das palavras-chaves para a Ficha Catalográfica.

dedicatória

`\dedicatoria` : ambiente para escrever a dedicatória. É possível trocar o espaçamento dentro desse ambiente do mesmo jeito que no $\text{\LaTeX}$ padrão.

agradecimentos

`\agradecimentos` : ambiente para escrever os agradecimentos. É possível trocar o espaçamento dentro desse ambiente do mesmo jeito que no $\text{\LaTeX}$ padrão.

resumo

`\chaves` : A lista das palavras chaves, separadas por ‘;’. Deve ser definido antes do ambiente `\resumo`, o qual é usado para escrever o resumo em português.

abstract

`\keys` : A lista das palavras chaves em inglês, separadas por ‘;’. Deve ser definido antes do ambiente `\abstract`, o qual contém 1 argumento e é usado escrever o resumo em inglês. O argumento deve ser o título do trabalho em inglês.

tabelas

`\tabelas` : Macro com 1 argumento opcional para gerar as tabelas. O argumento pode ser:

- nada [] : gera apenas o sumário;
- fig : gera o sumário e uma lista de figuras;
- tab : gera o sumário e uma lista de tabelas;
- alg : gera o sumário e uma lista de algoritmos;
- cod : gera o sumário e uma lista de programas.

Pode-se usar qualquer combinação dessas opções. Por exemplo:

- figtab : gera o sumário e listas de figuras e tabelas,
- figtabcod : gera o sumário e listas de figuras, tabelas e códigos de programas;
- figtabalg : gera o sumário e listas de figuras, tabelas e algoritmos;
- figtabalgcod : gera o sumário e listas de figuras, tabelas, algoritmos e códigos de programas

epígrafe

`\epigrafe` : Macro com 3 argumentos que permite editar um epígrafe. O primeiro argumento é o texto da citação. O segundo argumento é o nome do autor da citação. O terceiro argumento é o título da referência à qual a citação pertence.

A.3 Elementos do texto

A.3.1 Figuras

Rótulos de figuras e tabelas devem ser centralizados se tiverem até uma linha (Figura A.1), caso contrário devem estar justificados e identados em ambas as margens, como mostrado na Figura A.2. Essa formatação já é realizada automaticamente pela classe `inf-ufg`.

Os compiladores $\text{\LaTeX}$ provêem um mecanismo bastante simples para inclusão de figuras, o que pode ser feito com o auxílio de várias classes auxiliares (as mais comuns são `graphic` e `graphicx`). A classe `inf-ufg` usa o comando `\includegraphics`, da classe `graphicx`, para a inclusão de figuras e não é necessário você colocar a extensão do arquivo neste comando. Por exemplo, para a Figura A.1 os comandos usados foram:

```
\begin{figure}[htb]
\centering
\includegraphics[width=0.40\textwidth]{./fig/exemploFig1}
\caption{Uma figura típica.}
\label{fig:exemploFig1}
\end{figure}
```

Ao se usar o compilador $\text{\LaTeX}$, as figuras podem estar nos formatos *eps* e *ps*. Ao se usar o $\text{\PDFLaTeX}$, as figuras podem estar nos formatos *png*, *jpg*, *pdf* e *mps*. A classe `graphicx` também pode ser usada para a inclusão de figuras, nos formatos listados, ao se usar o $\text{\PDFLaTeX}$. Os comandos necessários são os mesmos ao se incluir figuras ao se usar o compilador $\text{\LaTeX}$. O uso do comando `\includegraphics` faz com que $\text{\PDFLaTeX}$ procure primeiro por figuras com extensão *pdf*, depois *jpg*, depois *mps* e por último *png*. Aqui também não é necessário especificar a extensão do arquivo.

Para a inclusão das figuras A.1 à A.3 os comandos usados, tanto no $\text{\LaTeX}$ quanto no $\text{\PDFLaTeX}$, seriam os mesmos. É claro que em cada caso devem estar disponíveis as figuras nos formatos suportados por cada compilador. Por exemplo, para a inclusão da Figura A.3 foram usados:

```
\begin{figure}[H]
\centering
\includegraphics[width=0.40\textwidth]{./fig/exemploFig3}
\caption{Figura incluída no texto com a classe graphicx.}
\label{fig:exemploFig3}
\end{figure}
```

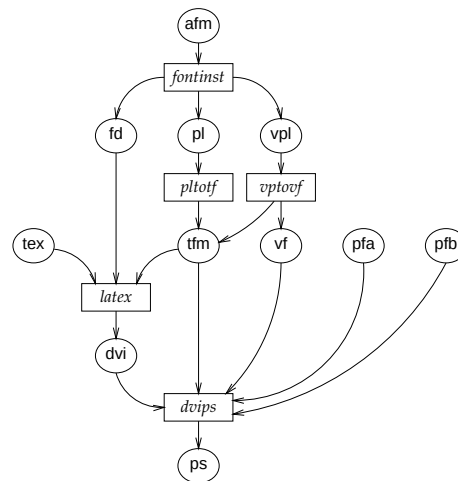


Figura A.1: Uma figura típica.

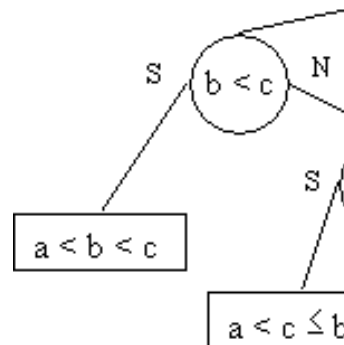


Figura A.2: Esta figura é um exemplo de um rótulo de figura que ocupa mais de uma linha, devendo ser indentado e justificado.

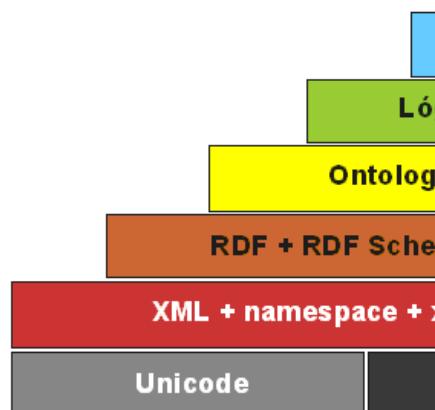
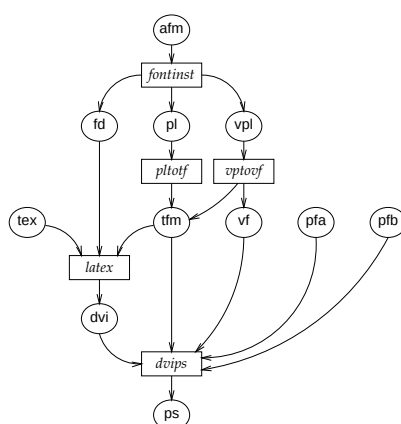


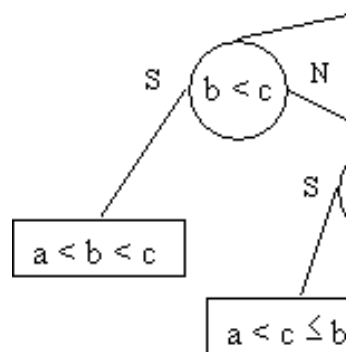
Figura A.3: Figura incluída no texto com a classe `graphicx`.

A.3.2 Subfiguras

A classe `subfigure` pode ser usada para a inclusão de figuras dentro de figuras (consulte a documentação da classe para maiores detalhes). Por exemplo, a Figura A.4 contém duas subfiguras. Estas podem ser referenciadas por rótulos independentes, ou seja, podem ser referenciadas como Figuras A.4(a) e A.4(b) ou Subfiguras (a) e (b).



(a) Primeira subfigura.



(b) Segunda subfigura (um pedaço).

Figura A.4: (a) e (b) representam dois exemplos do uso de subfiguras dentro de uma única figura.

A Figura A.4 foi incluída com os comandos listados a seguir. Observe que há rótulos independentes para cada uma das subfiguras e um rótulo geral para a figura, os quais podem ser todos referenciados.

```
\begin{figure}[h]
\centering
\subfigure[Primeira subfigura.]
{
\includegraphics[width=0.35\textwidth]{./fig/exemploFig1}
\label{subfig:ex1}
} \quad
\subfigure[Segunda subfigura (um pedaço).]
{
\includegraphics[width=0.30\textwidth]{./fig/exemploFig2}
\label{subfig:ex2}
}
\caption{{\subref{subfig:ex1}} e {\subref{subfig:ex2}} representam
dois exemplos do uso de subfiguras dentro de uma única
figura.}
\label{fig:subfiguras}
\end{figure}
```

Caso uma subfiguras não tenha rótulo, para evitar que o apenas o número da mesma apareça na Lista de Figuras, use o comando `\subfigure[] []`. Caso uma subfigura tenha rótulo e deseja-se evitar que a mesma apareça na Lista de Figuras, use o comando `\subfigure[] [Rótulo]`.

A.3.3 Tabelas

Em tabelas, deve-se evitar usar cor de fundo diferente do branco e o uso de linhas grossas ou duplas. Ao relatar dados empíricos, não se deve usar mais dígitos decimais do aqueles que possam ser garantidos pela sua precisão e reprodutibilidade. Rótulos de tabelas devem ser colocados antes das mesmas (veja a Tabela A.1).

A.3.4 Algoritmos

Algoritmos devem ser representados no formato do Algoritmo A.1, que foi descrito com o uso da classe `algorithm2e`. A rigor não é obrigatório o uso dessa classe, contudo o uso da mesma permite que seja gerada automaticamente uma lista de algoritmos logo após o sumário.

Tabela A.1: *Conteúdo do diretório*

Tag	Comprimento	Início	Tag	Comprimento	Início
001	0020	00000	100	0032	00235
003	0004	00020	245	0087	00267
005	0017	00024	246	0036	00354
008	0041	00041	250	0012	00390
010	0024	00082	260	0037	00402
020	0025	00106	300	0029	00439
020	0044	00131	500	0042	00468
040	0018	00175	520	0220	00510
050	0024	00193	650	0033	00730
082	0018	00217	650	0012	00763

Algoritmo A.1: $MSR(A, i, j)$

Entrada: vetor $A[i..j]$, inteiros não negativos i e j .**Saída:** vetor $A[i..j]$ ordenado.

```

1  $n \leftarrow j - i.$ 
2 se ( $n < 4$ ) então
3   | Ordene com  $\leq 3$  comparações.
4 senão
5   | Divida  $A$  em  $\lceil \sqrt{n} \rceil$  subvetores de comprimento máximo  $\lfloor \sqrt{n} \rfloor$ .
6   | Aplique  $MSR$  a cada um dos subvetores.
7   | Intercale os subvetores.
8 fim
```

A.3.5 Códigos de Programa

Códigos de programa podem ser importados, mantendo-se a formatação original, conforme se pode ver no exemplo do Código A.1. Este exemplo usa o ambiente `codigo`, definido na classe `inf-ufg`, que permite que uma lista de programas seja gerada automaticamente logo após o sumário.

Código A.1 insertionsort()

```

1 void insertionSort( int* v, int n )
2 {
3     int i    = 0;
4     int j    = 1;
5     int aux = 0;
6
7     while (j < n)
8     {
9         aux = v[j];
10        i   = j - 1;
11        while ((i >= 0) && (v[i] > aux))
12        {
13            v[i + 1] = v[i];
14            i = i - 1;
15        }
16        v[i + 1] = aux;
17        j = j + 1;
18    }
19 }

```

A.3.6 Teoremas, Corolários e Demonstrações

O uso do ambiente theorem permite a escrita de teoremas, como no exemplo a seguir:

```
\begin{theorem}[Pitágoras]
```

Em todo triângulo retângulo o quadrado do comprimento da hipotenusa é igual a soma dos quadrados dos comprimentos dos catetos.

```
\end{theorem}
```

O resultado é o mostrado a seguir:

Teorema A.1 (Pitágoras) *Em todo triângulo retângulo o quadrado do comprimento da hipotenusa é igual a soma dos quadrados dos comprimentos dos catetos.*

Da mesma forma pode-se usar o ambiente proof para demonstrações de teoremas:

```
\begin{proof}
```

Para demonstrar o Teorema de Pitágoras \dots

```
\end{proof}
```

Neste caso, o resultado é:

Prova. Para demonstrar o Teorema de Pitágoras ...

□

Além desses dois ambientes, estão definidos os ambientes `definition` (Definição), `corollary` (Corolário), `lemma` (Lema), `proposition` (Proposição), `comment` (Observação).

A.3.7 Citações Longas

Segundo as normas da ABNT, uma citação longa (mais de 3 linhas) deve seguir uma formação especial. Para tanto foi criado o ambiente `citacao`, o qual é baseado no ambiente de mesmo nome definido pelo grupo ABNTEX [Santini Frasson, 2002]:

Uma citação longa (mais de 3 linhas) deve vir em parágrafo separado, com recuo de 4cm da margem esquerda, em fonte menor, sem as aspas [ABNT, 2001a, 4.4] e com espaçamento simples [ABNT, 2001b, 5.3]. Uma regra de como fazer citações em geral não é simples. É prudente ler [ABNT, 2001a] se você optar por fazer uso frequente de citações. Para satisfazer às exigências tipográficas que a norma pede para citações longas, use o ambiente `citacao`.

Este exemplo de citação longa foi produzido com o uso do ambiente `citacao`, como descrito logo a seguir:

```
\begin{citacao}
Uma citação longa (mais de 3 linhas) deve vir em parágrafo
separado, com recuo de 4cm da margem esquerda, em fonte menor,
sem as aspas \cite[4.4]{NBR10520:2001} e com espaçamento
simples \cite[5.3]{NBR14724:2001}. Uma regra de como fazer
citações em geral não é simples. É prudente ler
\cite{NBR10520:2001} se você optar por fazer uso frequente
de citações. Para satisfazer às exigências tipográficas que a
norma pede para citações longas, use o ambiente citacao.
\end{citacao}
```

A.3.8 Referências Bibliográficas

Esta seção mostra exemplos de uso de referências bibliográficas com BIBTEX e do comando `\cite`. As entradas listadas na página 26 são obtidas com esses recursos. Um grande repositório de referências já em formato BIBTEX está disponível em: <http://www.math.utah.edu/~beebe/bibliographies.html>.

As referências bibliográficas devem ser não ambíguas e uniformes. Recomenda-se usar números entre colchetes, como por exemplo [Smith & Jones, 1999], [Knuth, 1989]

e [Berners-Lee & Swick, 2002]. O comando `\nocite` não produz texto, mas permite que a entrada seja incluída nas referências. Por exemplo, o comando `\nocite{Ber1970}` gera na lista de referências bibliográficas a entrada referente à chave `Ber1970`, mas não inclui nenhuma referência no texto. O comando `\nocite{*}` faz com que todas as entradas do arquivo de dados do `BIBTEX` sejam incluídas nas referências.

Existem vários livros sobre `LATEX`, como [Hahn, 1991, Kopka & Daly, 1995], embora os mais famosos sejam sem dúvida [Lamport, 1996] e [Goossens et al., 1994]. Para converter documentos `LATEX` para HTML veja [Drakos, 1994, pg.1–10].